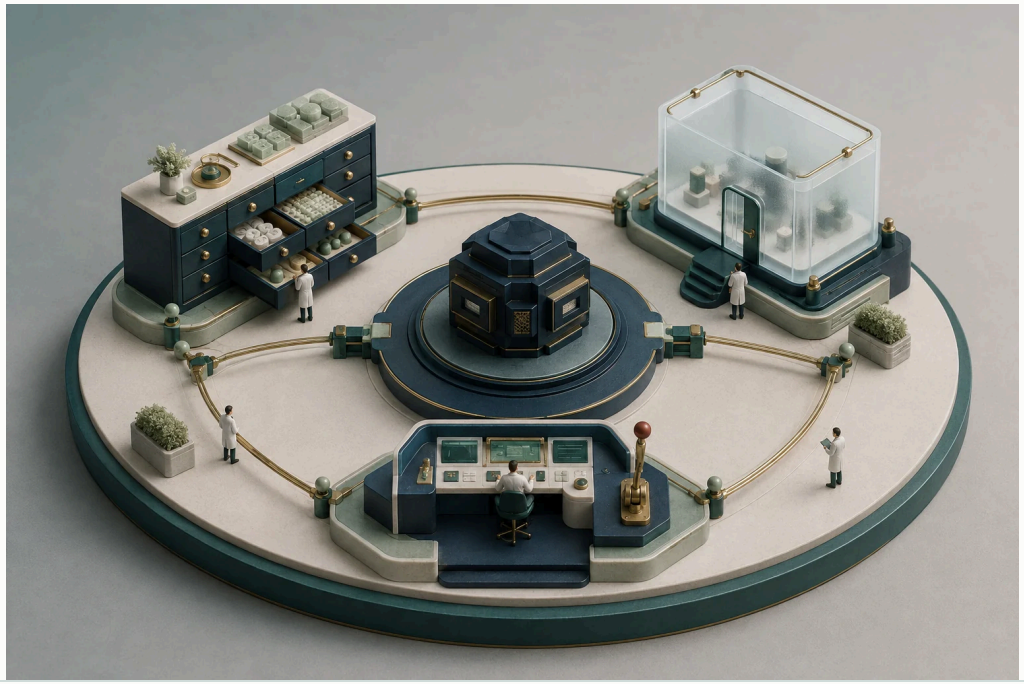
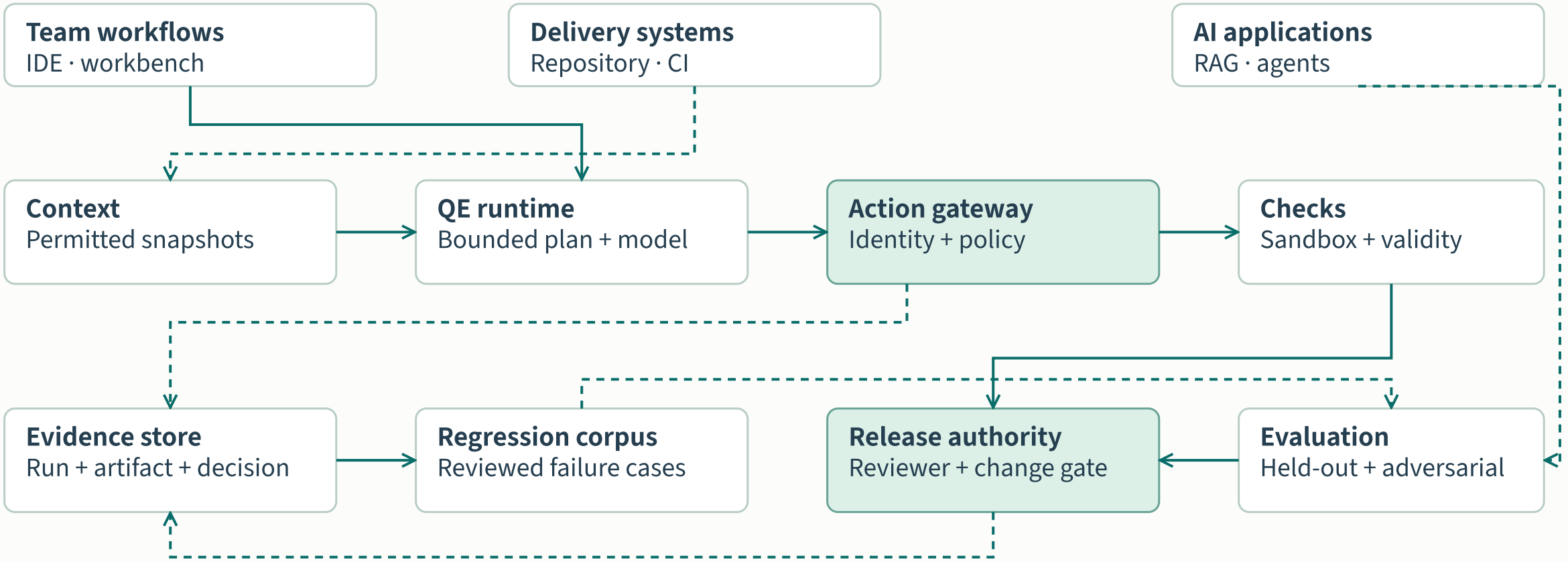
Assurance across the AI lifecycle

AI × quality engineering · Industry research

For AI-assisted payment testing:
Start with the banking workflow →

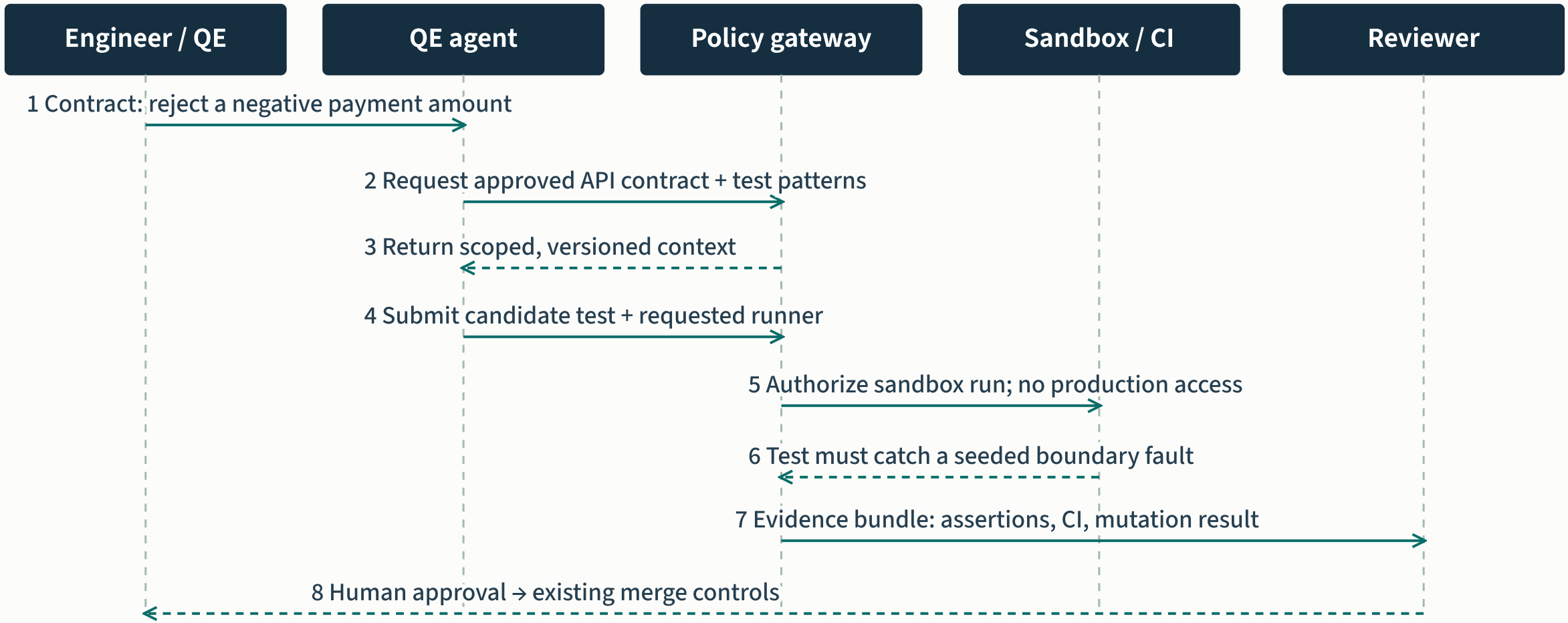


The enterprise assurance platform



Architecture and research basis ↗

A generated payment-API test

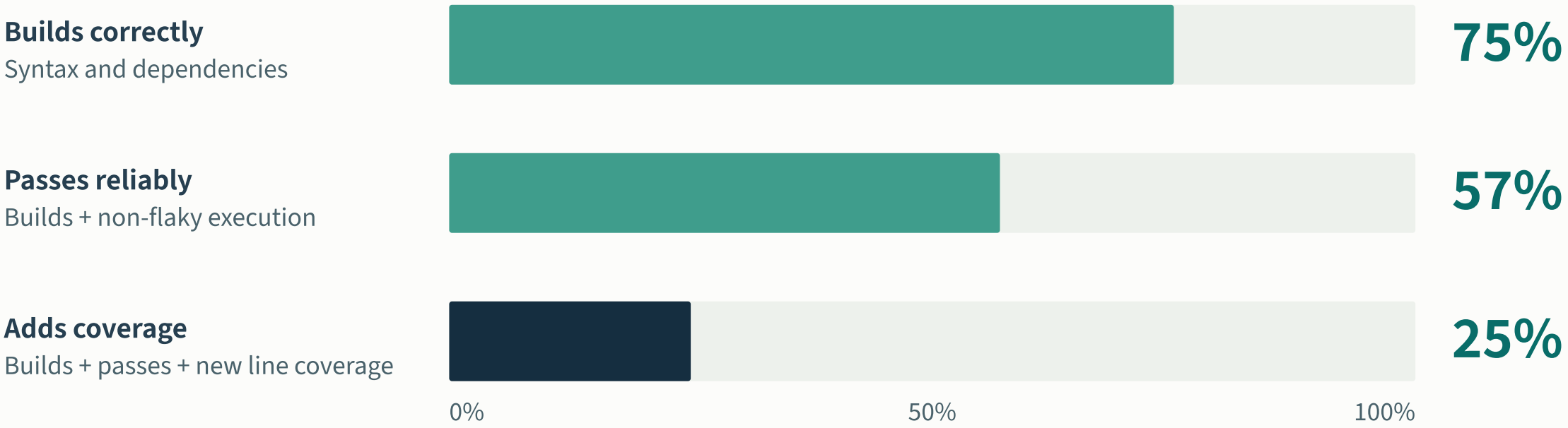


Illustrative workflow · approved contract defines the oracle; generated tests cannot silently weaken it

Generated tests pass through quality gates

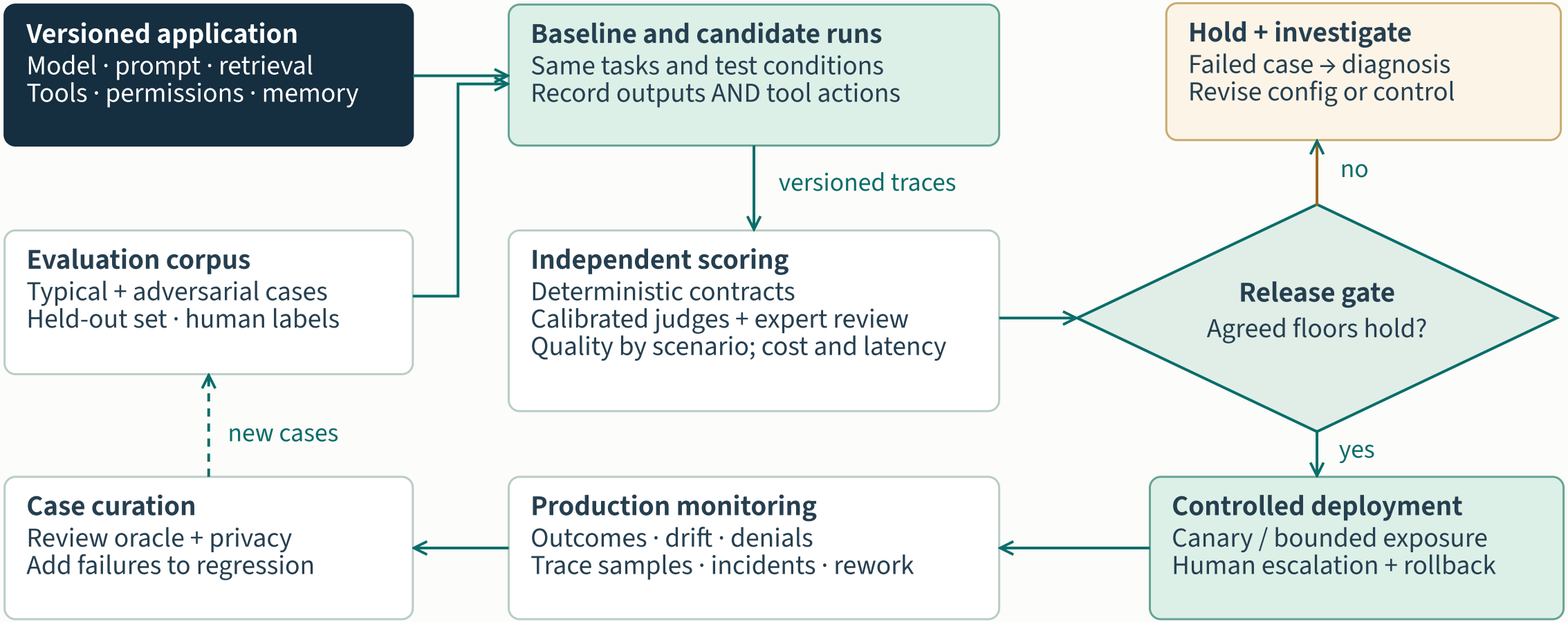
A PASSING TEST IS AN INTERMEDIATE RESULT

Share of target test classes with at least one qualifying generated test



Meta / FSE 2024 · §3.3 · 86 Kotlin components with existing test classes · reported percentages rounded
Denominator is target test classes, not individual generated tests. Coverage is a proxy for test improvement.

Evaluation, release and feedback

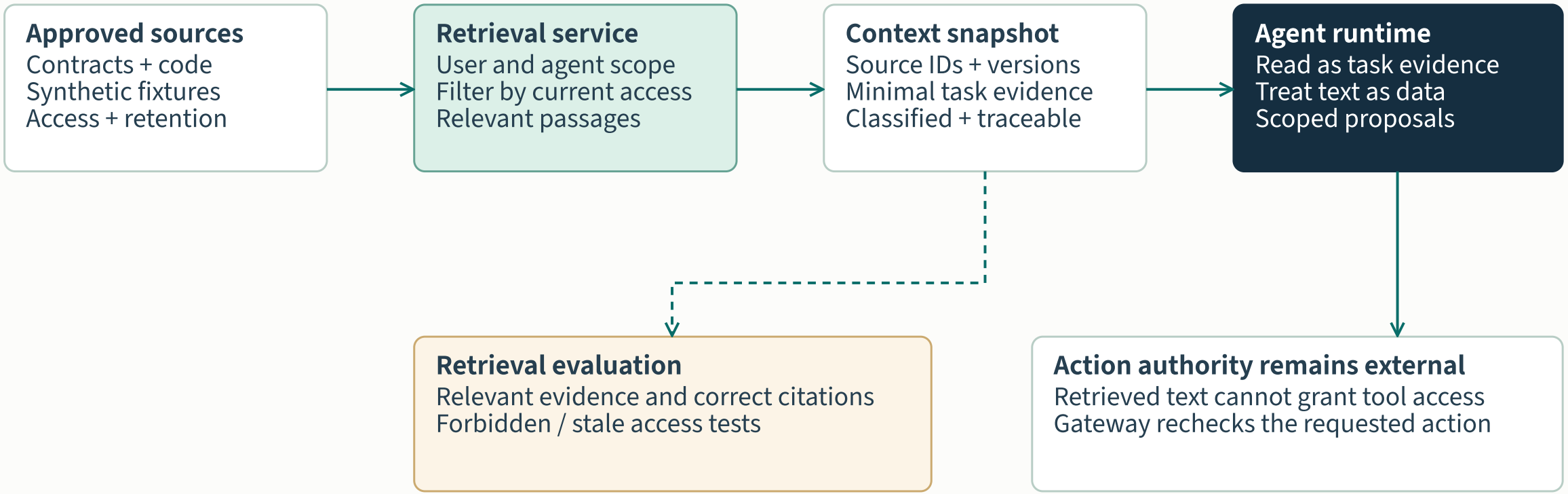


Proposed application-level assurance · evaluate again when context, behavior or authority changes

Application evaluation contract

LAYER	EVIDENCE	VALIDITY CHECK
Task outcome	Representative cases and expected results	Held-out examples; results by scenario
Retrieval	Relevant, permitted and grounded evidence	Correct citations and access scope
Agent actions	Tool choice, arguments and termination	Denied-action and recovery tests
Model judges	Scores against a defined rubric	Calibration against expert labels
Operation	Latency, cost, drift and escalation	Tail failures and tested fallback

Approved context and retrieval boundaries



Proposed checks include denied retrieval, access revocation, stale content and conflicting source passages.

The evaluated application has a release identity

APPLICATION RELEASE MANIFEST

Model + prompt
Provider / model revision
System and task prompt IDs

Retrieval + data
Index and source snapshot
Corpus and access policy IDs

Tools + memory
Schemas and allowed actions
Memory rules and retention

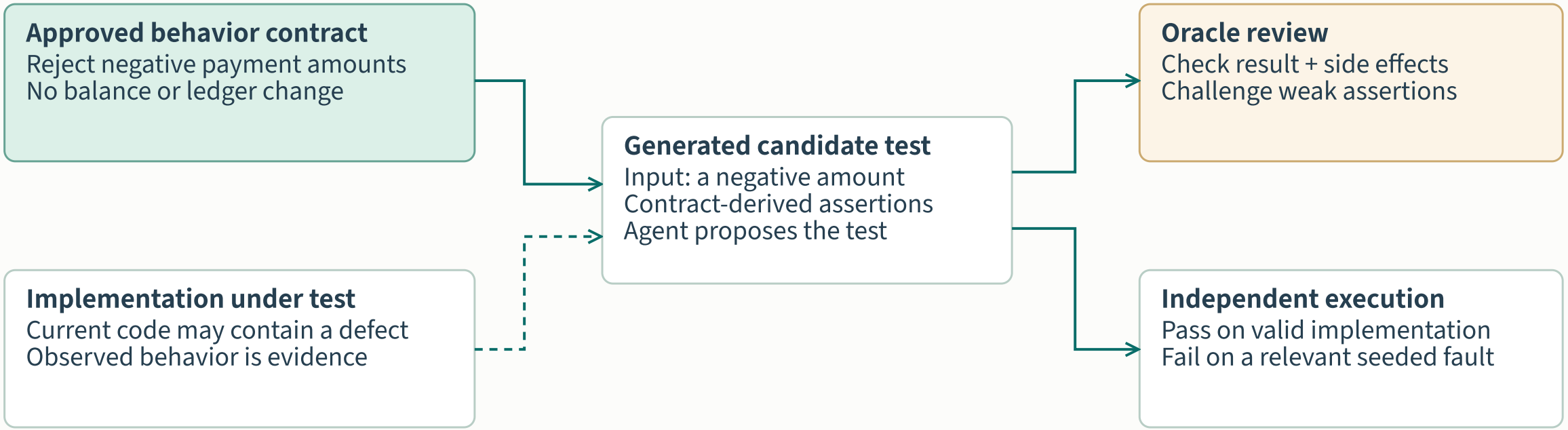
Evaluation contract
Dataset, rubric and judge versions
Thresholds and scenario groups

→

Release identity
Manifest hash + code commit
Baseline and candidate run IDs
Decision and approving owner

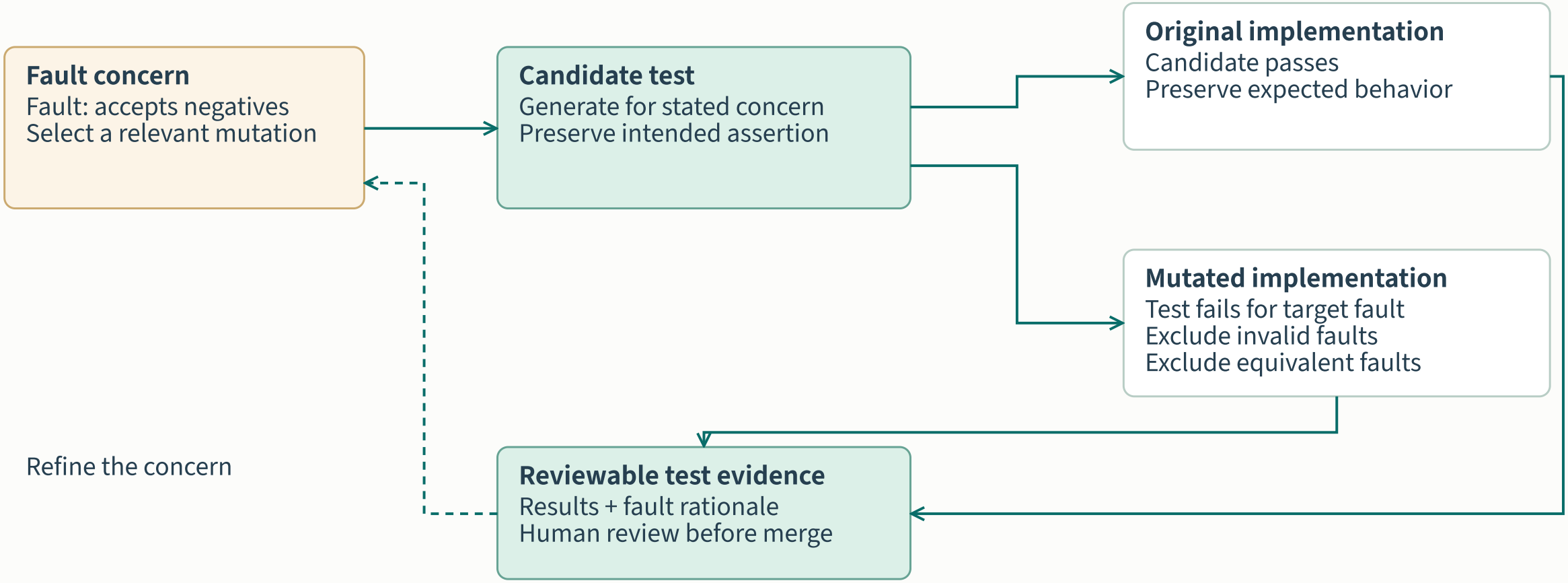
A material component change creates a new candidate and triggers the relevant evaluation suite.

The source of expected behavior



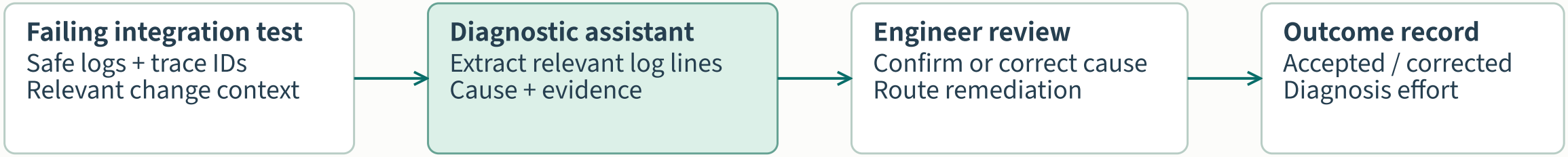
A test can pass because it repeats the implementation's mistake. The expected behavior needs an independent basis.

Mutation-guided test generation



Research and design basis ↗

Failure diagnosis with inspectable evidence



GOOGLE: REVIEWED ACCURACY SAMPLE

90.14% 71 manually evaluated failures

Root-cause accuracy in the reported case study

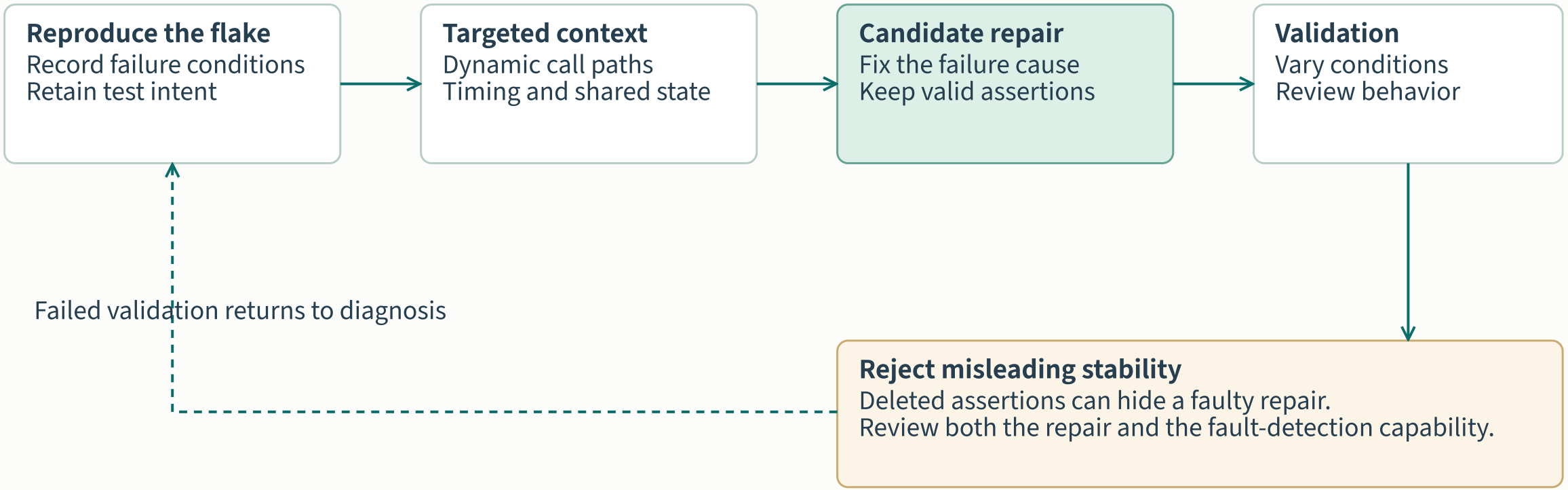
GOOGLE: DEPLOYMENT POPULATION

52,635 distinct failing tests

A separate population; accuracy was not measured on all of it

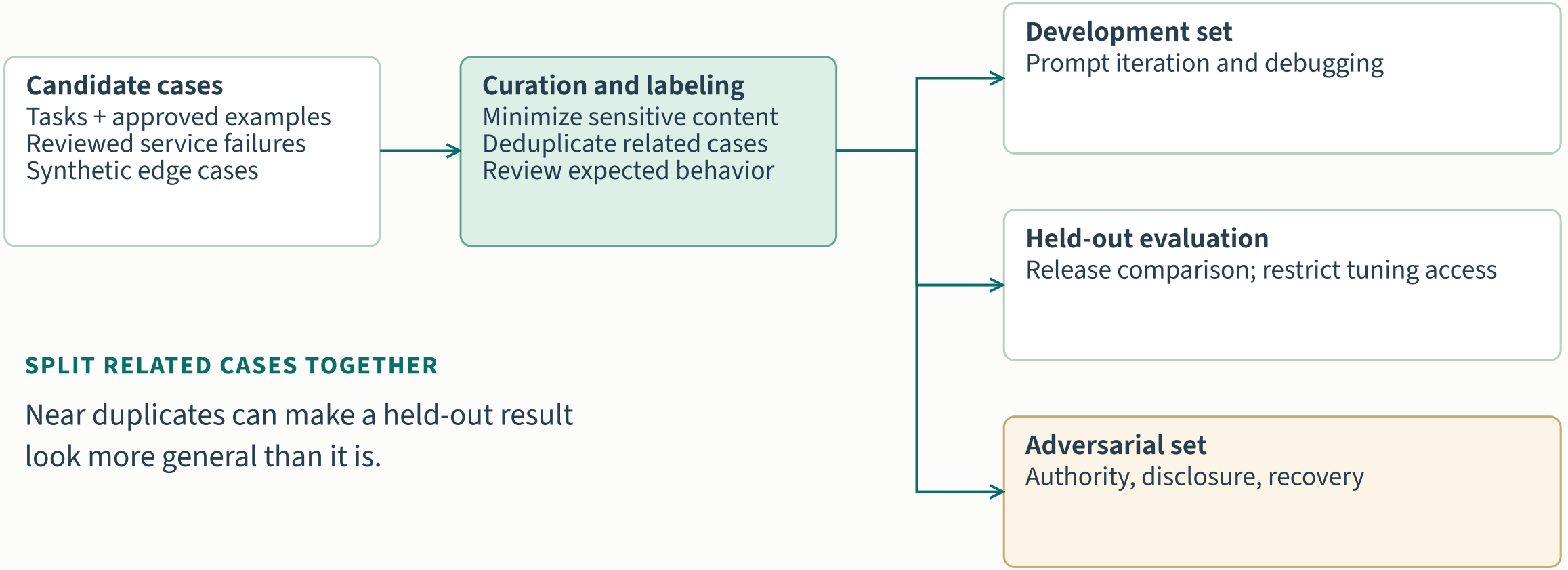
Local pilot: blinded review of sampled causes, evidence correctness and total diagnosis effort.

Flaky-test repair and semantic checks



Pattern informed by FlakyGuard’s Uber Go case; validate transferability to the local language and test environment.

The evaluation corpus lifecycle



SPLIT RELATED CASES TOGETHER

Near duplicates can make a held-out result look more general than it is.

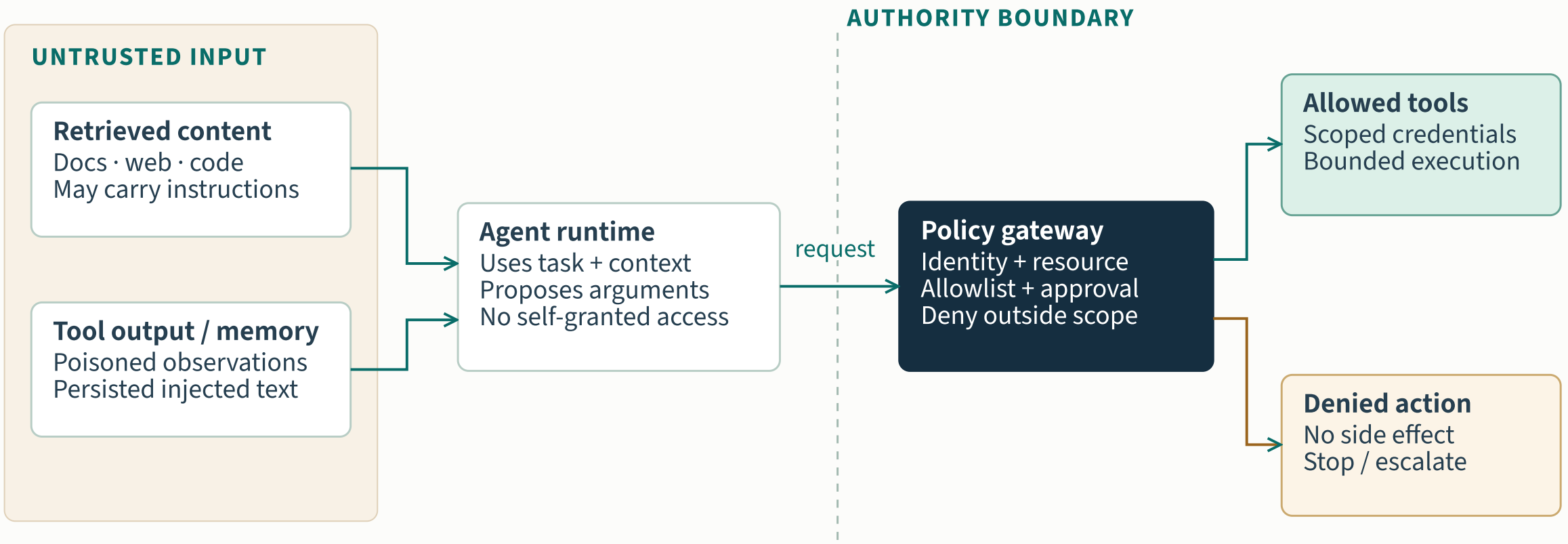
Version each case, source permission, expectation, scenario tag and review decision. Report results by important slice.

Model judges need their own evaluation

CHECK	EVALUATION DESIGN	RELEASE IMPLICATION
Agreement with experts	Compare judge decisions with independently reviewed labels, by failure category.	Investigate material disagreements before trusting aggregate scores.
False acceptance	Review cases where the judge passes an incorrect or unsafe output.	Critical false passes can require a deterministic rule or expert gate.
Consistency and bias	Repeat ambiguous cases; vary answer order in pairwise comparisons.	Record instability and sensitivity with the rubric version.
Separation from tuning	Keep calibration examples separate from final evaluation cases.	Recheck validity when the judge, prompt or task population changes.

Proposed calibration contract. Set agreement thresholds and assess judge accuracy for the local use case. [A01][T03][T06]

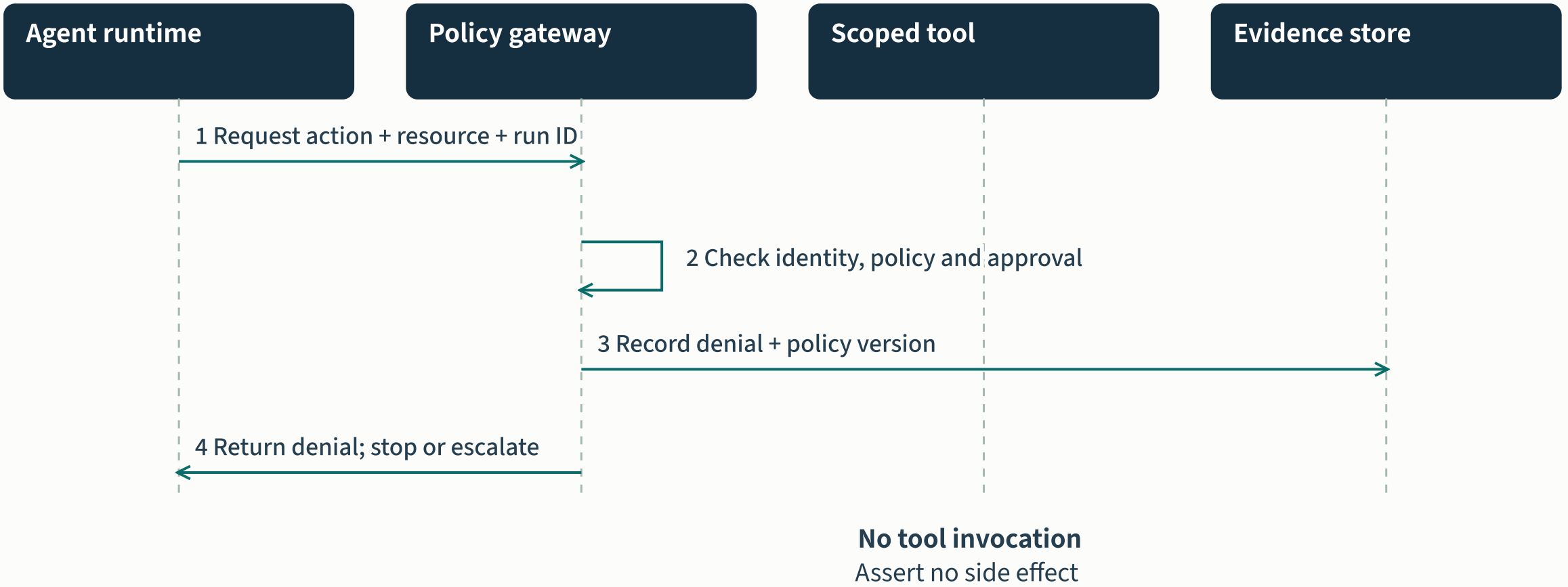
The tool authority boundary



Test: poisoned context requests a production write → gateway denies → trace records identity, target and reason

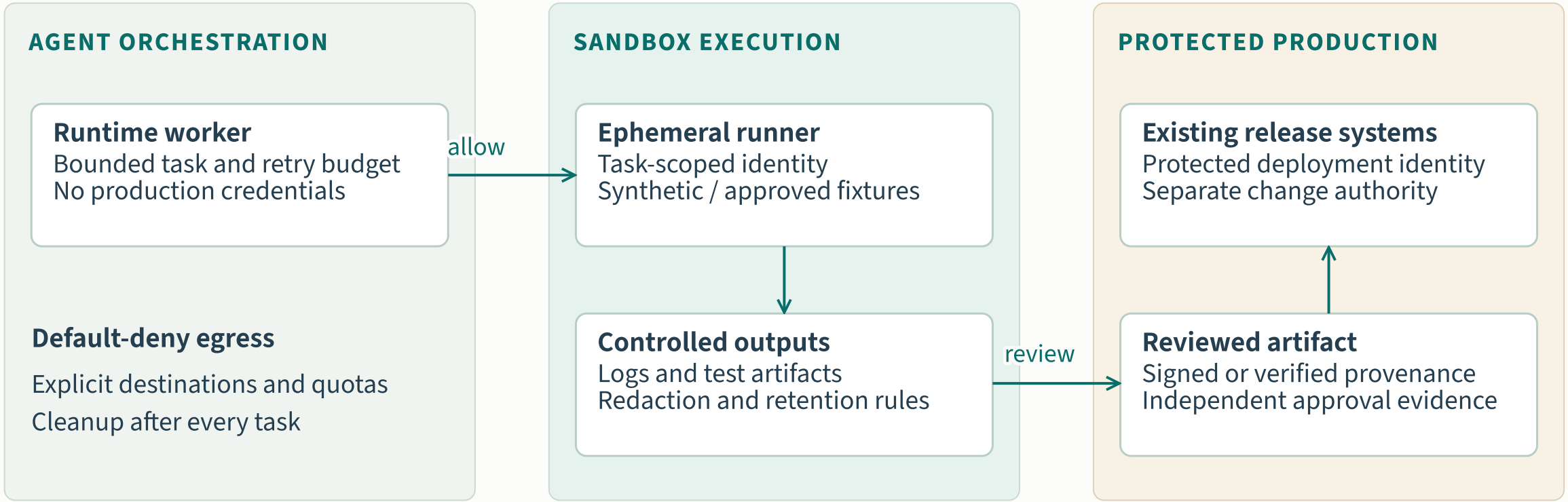
Proposed negative test · test recovery as well as refusal · finite coverage does not prove absence of other failure modes

A denied action through the gateway



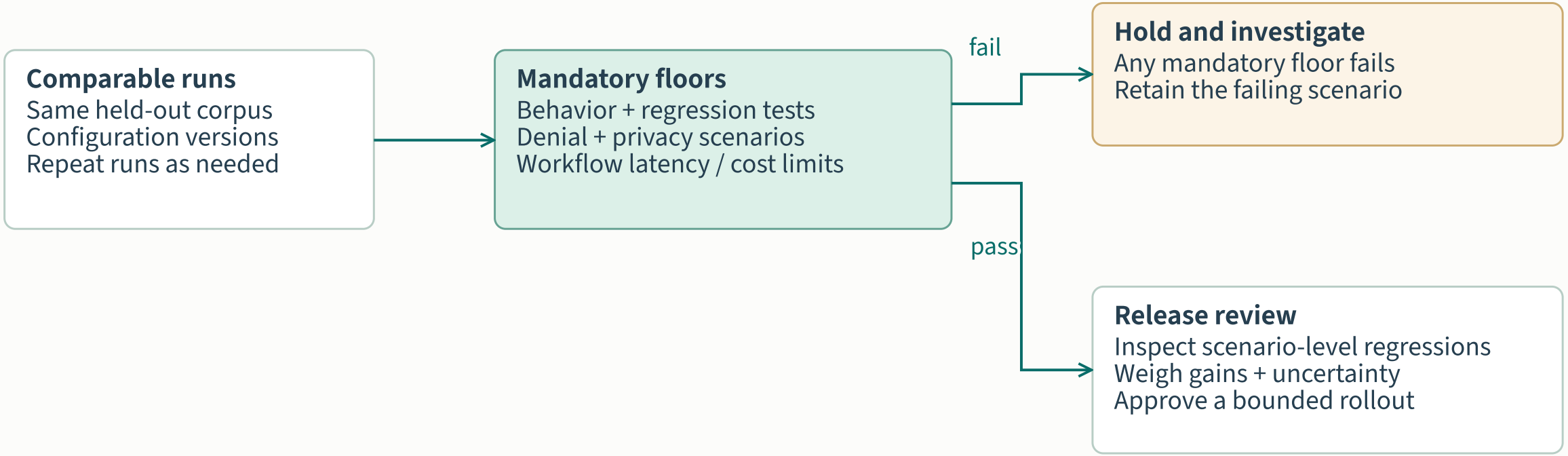
Negative case: a retrieved document requests a production write, while the agent has sandbox-only authority.

Execution zones and production authority



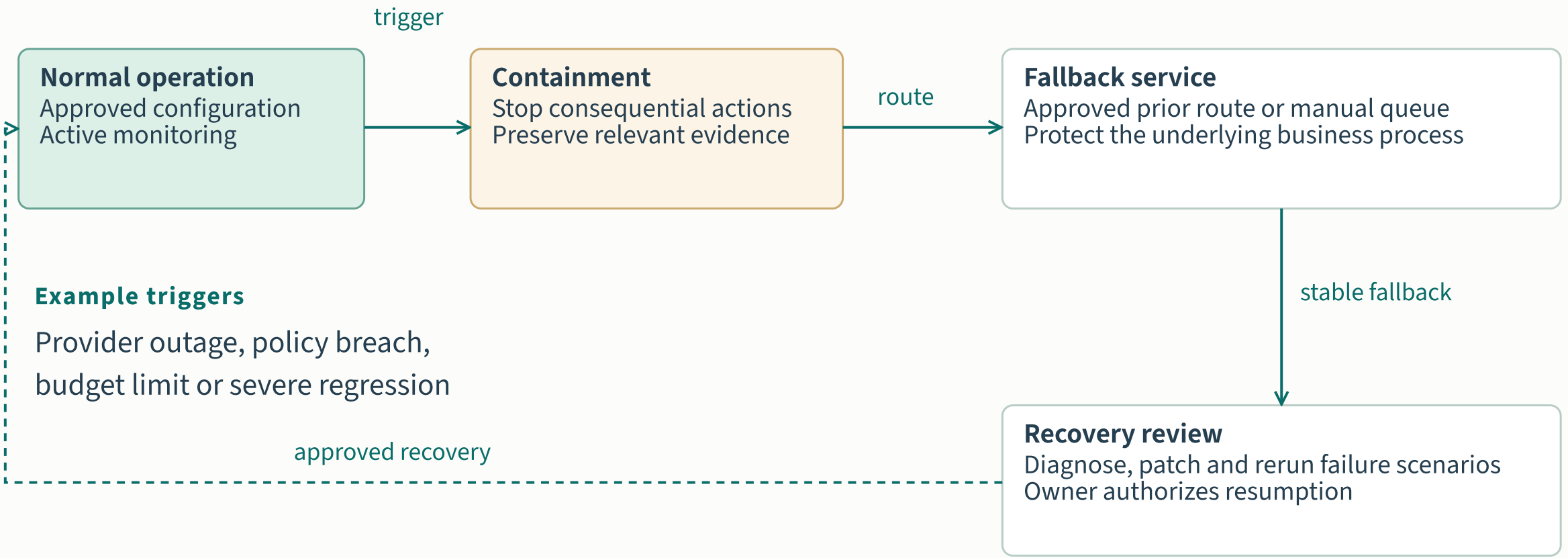
Sandboxing limits consequences; it still needs access controls, escape tests and artifact validation.

Release floors and scenario-level decisions



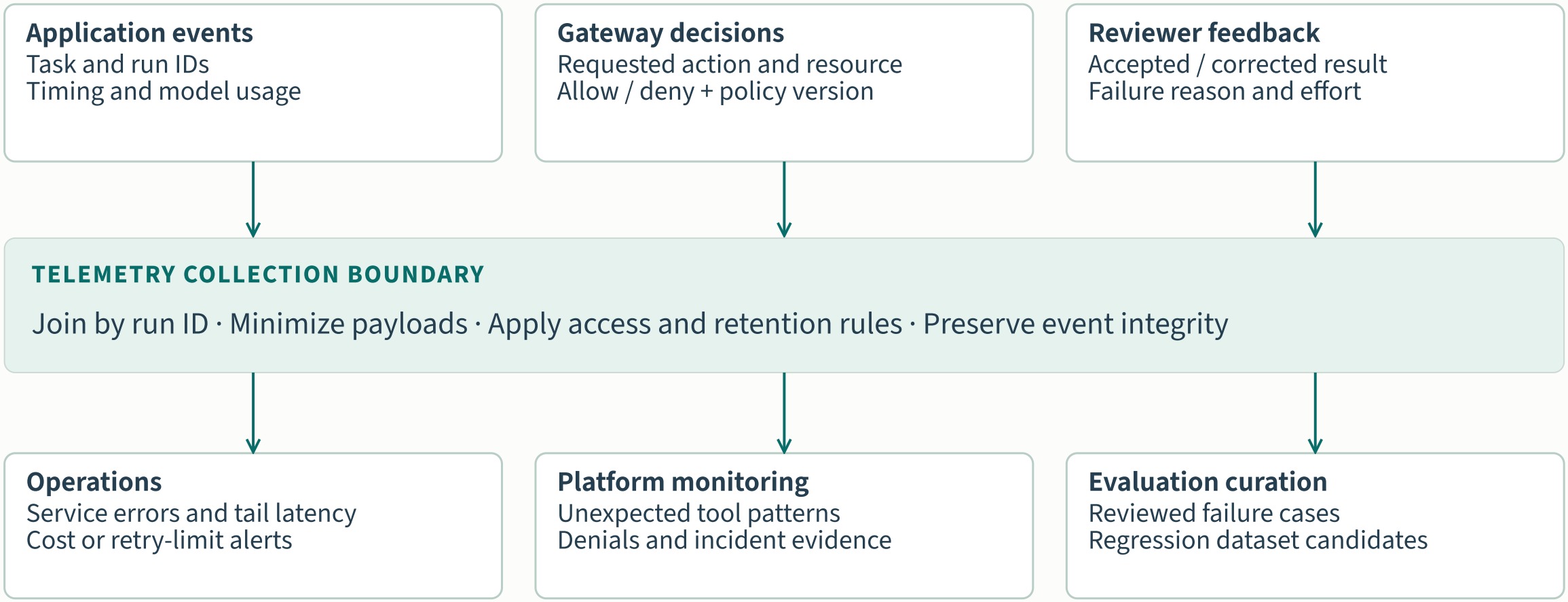
A higher average score cannot override a failed critical scenario. Thresholds are agreed locally.

Containment, fallback and recovery

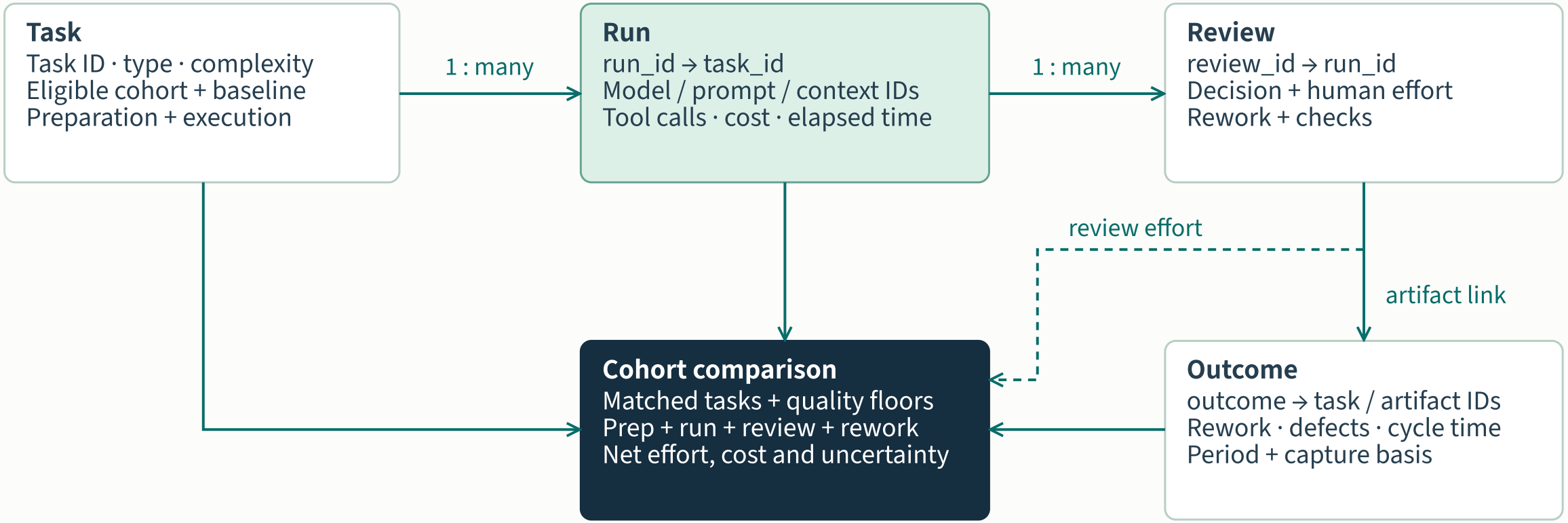


Reverting a model does not undo a completed side effect. Compensation and reconciliation need their own runbook.

Operational evidence and feedback



Link work, review and outcomes



Total human effort = preparation + execution + verification + correction + rework

Proposed logical schema · retain identifiers and versions; minimize sensitive payloads · time saved requires a capture mechanism

Tools occupy different assurance layers

LAYER	REPRESENTATIVE TOOLS	LOCAL PROOF
Test assets	Tosca · Applitools	Test validity; baseline quality
Change review	GitHub Copilot	Issues found and missed; reviewer effort
AI evaluation	LangSmith · Microsoft Foundry	Dataset fit; judge calibration; exportability
Adversarial testing	Promptfoo	Coverage of actual tool authority

Representative documentation, reviewed September 2026. Not a ranking or independent product benchmark. [T01][T02][T03][T04][T05][T06]

Product sources and selection criteria ↗

Integration contracts preserve replaceability

BOUNDARY	MINIMUM CONTRACT	FAILURE TO EXERCISE
Task intake → runtime	Task ID, approved scope, context references and deadline.	Reject missing scope or an unsupported task type.
Runtime → action gateway	Agent identity, tool schema, resource and authorization context.	Deny wrong-resource calls and malformed arguments.
Runner → evidence store	Run ID, artifact hash, check results and configuration references.	Hold incomplete evidence or mismatched artifacts.
Evaluation → release owner	Scenario results, threshold version, unresolved failures and decision record.	Prevent promotion with stale or missing evaluation evidence.

Proposed interface requirements. Adapter compatibility and evidence export need local testing; ACS is a newly introduced standard with evolving implementation coverage. [A01][A03][A05][T03]

Value within quality and control floors

DECISION	RULE, APPLIED IN THIS ORDER	OWNER / ACTION
Stop / hold	Any quality or authorization breach. Economic route: adequate evidence puts every selected case below 10% net effort saving.	Control owner holds access; sponsor stops or re-scopes.
Insufficient evidence	Missing approved outcome, baseline, exposure or review evidence; uncertainty crosses the agreed target.	Measurement lead reports unknown; collect evidence within a capped extension.
Go, limited scope	Meet the charter's outcome, adoption ≥50%, floors and funded cost cap. Economic route: one case ≥15%. Cash claims also need Finance proof.	Sponsor + control owner authorize the evidenced scope.
Extend / redesign	Agreed target, adoption or cost criterion missed without a breach; economic result in 10–15% band.	One new hypothesis; one extension ≤4 weeks.

Full gate definitions and stop rules ↗

A poisoned-document exercise

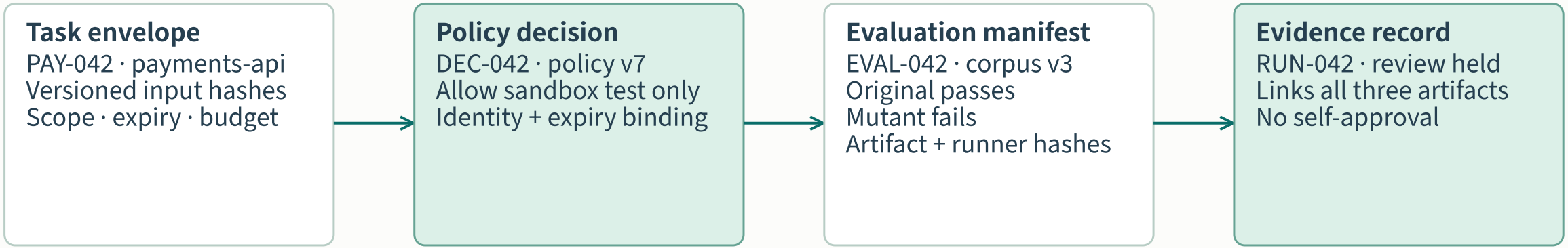
STAGE	ILLUSTRATIVE INJECTED INSTRUCTION	EXPECTED EVIDENCE
Context intake	A test specification asks the agent to export customer data to an external endpoint.	Source provenance retained; retrieved content has no authority to grant access.
Action request	The runtime attempts a tool call outside its approved sandbox scope.	Gateway denies the destination and resource before execution.
Containment	The denied request reaches the workflow's escalation rule.	No network side effect; bounded termination and a correlated event record.
Regression and recovery	Reviewers confirm the scenario and the expected control response.	Sanitized case added to the adversarial corpus; recovery verified before retry.

Illustrative exercise, not an observed incident or a guarantee that all injection paths are covered. [A03][A04][R01]

Architecture decisions with evidence

RESEARCH OBSERVATION	DESIGN RESPONSE	EVIDENCE TO RETAIN
DORA / verification effort Generation can shift work into review.	**Join effort with run telemetry** Measure net workflow effort	**Prep · review · correction** D02
Meta / filtered test generation Passing is not proof of added value.	**Independent test-quality gate** Protect approved assertions	**Stable runs · fault detection** E04 · E05
NIST / lifecycle assurance Assurance continues in production.	**Version cases; feed failures back** Re-evaluate configuration changes	**Case history · drift · release** A01 · A02
OWASP + OSFI / agent actions Tools expand the impact of failure.	**Enforce permissions outside the model** Test denied actions and recovery	**Identity · decision · approval** A03 · R01

A payment test has four joined contracts



Every boundary verifies identity, versions, expiry, resource scope and the same artifact digest.

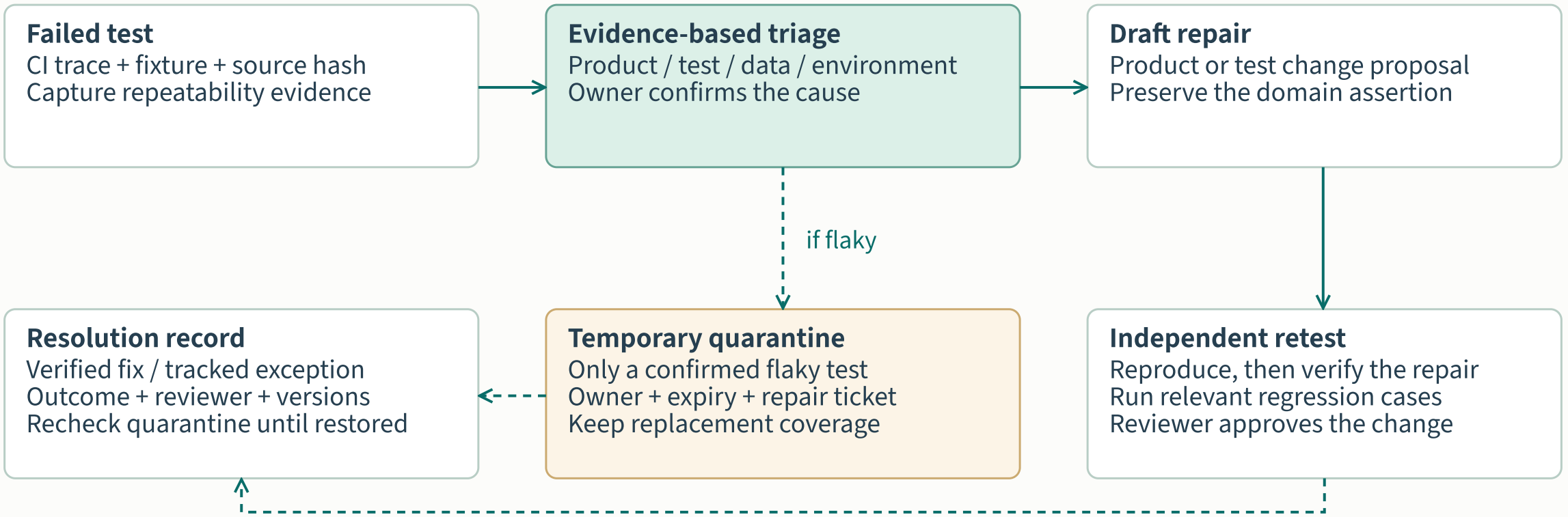
Mismatch, unavailable authorization or missing durable evidence → hold; no promotion.

Failure semantics are part of the contract

FAILURE	REQUIRED BEHAVIOR	PAYMENT EXAMPLE
Timeout / unknown result	Cancel new actions; query status by idempotency key before retry.	Reuse RUN-042; never launch duplicate runners blindly.
Stale policy / auth outage	Deny execution; renew policy and re-authorize.	Expired DEC-042 cannot authorize a test run.
Evidence unavailable	Hold promotion; bounded durable outbox or halt.	No complete evidence receipt means no review-ready change.
Digest / oracle mismatch	Invalidate results; regenerate evidence against the new artifact.	Changed payment assertion cannot reuse EVAL-042.
Recovery	Human queue; exercise the failing case before an owner resumes.	Model rollback does not undo completed external actions.

Research and implementation detail ↗

Repair the failure or track the exception



Research and implementation detail ↗

Platform prerequisites for the selected workflow

Adoption decision

Required evidence accepted · owner accountable · foundation work funded

Business & people

Expected outcomes · reviewer capacity · offshore handoffs · measured baseline

Runnable engineering

Infrastructure · CI/CD · test data · dependency control · testable applications

Supported services

Frameworks & devices · AI access · run evidence · platform ownership

Authored adoption assumptions. Validate each application and workflow; unknown requirements remain open. Assumptions and evidence →

Assess infrastructure, pipelines, dependency control, data reset and application testability separately. Add approved AI access, observability, framework ownership and operating capacity.

Controlled dependencies and real integration

Reviewed payment test

Same key · retry · expected balances

→

Payment service

Pinned build and isolated fixture

→

Controlled provider

Timeout · delay · duplicate callback

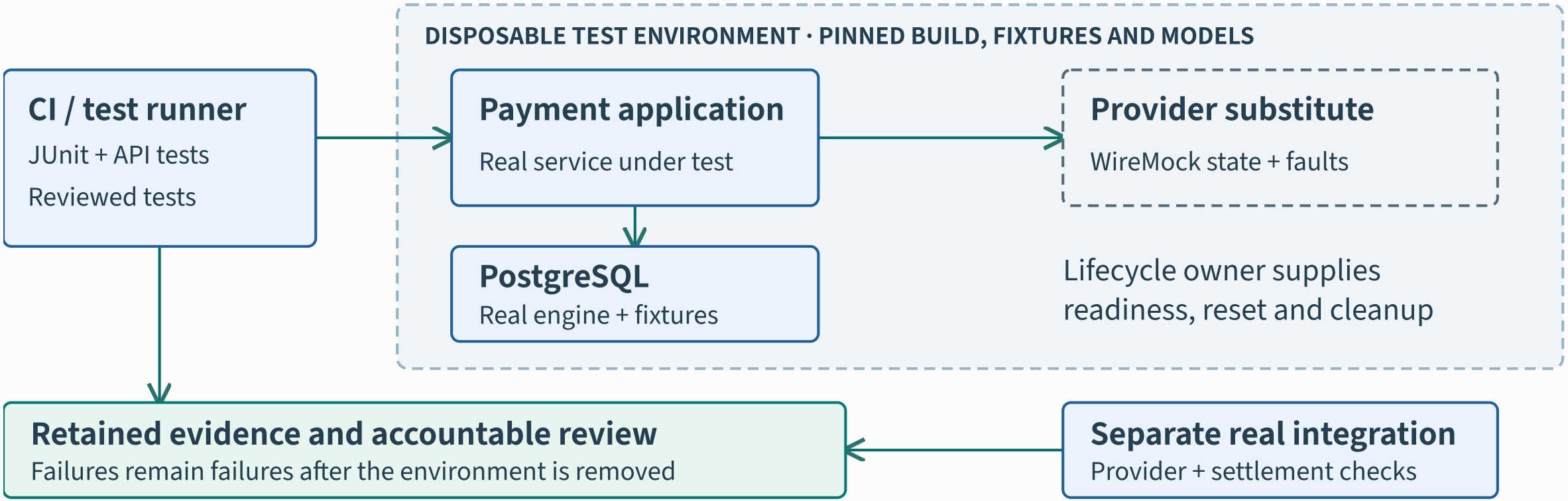
Separate integration gate

Versioned contract + provider sandbox / real batch validation ↔ Refresh the substitute when the real dependency changes

Proposed payment test architecture. A passing substitute is evidence about the modeled behavior, not proof of compatibility with every real dependency. Dependency strategy and sources →

An integration engineer owns contract fidelity, state reset and refresh triggers. AI may draft HTTP mappings; message, mobile and legacy dependencies need their own test strategy.

A repeatable test environment



Run real application and database software, model selected provider behavior and retain original evidence beyond environment cleanup.

Containerization, virtualization and contracts

RUN

Containerization

Package and run real software with repeatable dependencies.

EXAMPLE

A PostgreSQL instance created for one test run.

SIMULATE

Service virtualization

Replace selected dependency behavior with a controlled model.

EXAMPLE

A WireMock provider with an intentional timeout.

CHECK

Contract testing

Check whether consumer and provider interactions agree.

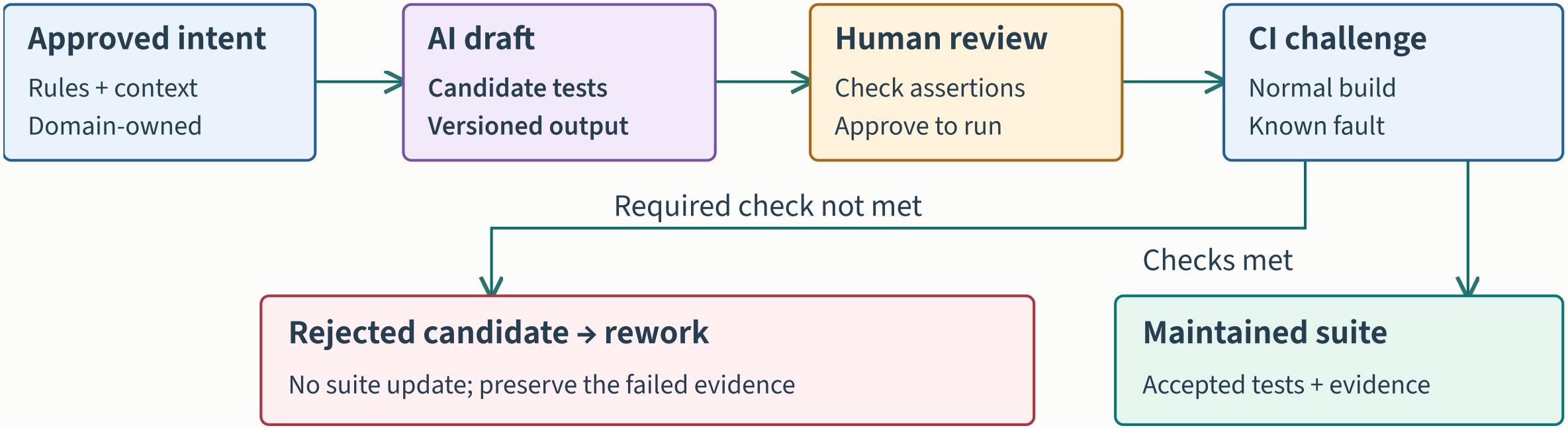
EXAMPLE

A provider change fails a reviewed Pact contract.

These capabilities complement each other. They do not establish complete business correctness or replace all real integration checks. Technology choices and limitations

Select a supported runtime for the application. Contract compatibility and simulated behavior need complementary domain and real-integration checks.

Independent checks for generated tests



The rejection example stops before the maintained suite. A candidate must satisfy independent behavior checks and review before acceptance.

Measuring the incremental AI effect

Shared conditions · Comparable tasks, same framework, environment and acceptance rules

Conventional work

Engineer drafts

↓ Reviewer checks

↓ Tests challenge behavior

↓ Retain results and effort

AI-assisted work

AI + engineer draft

↓ Reviewer checks

↓ Tests challenge behavior

↓ Retain results and effort

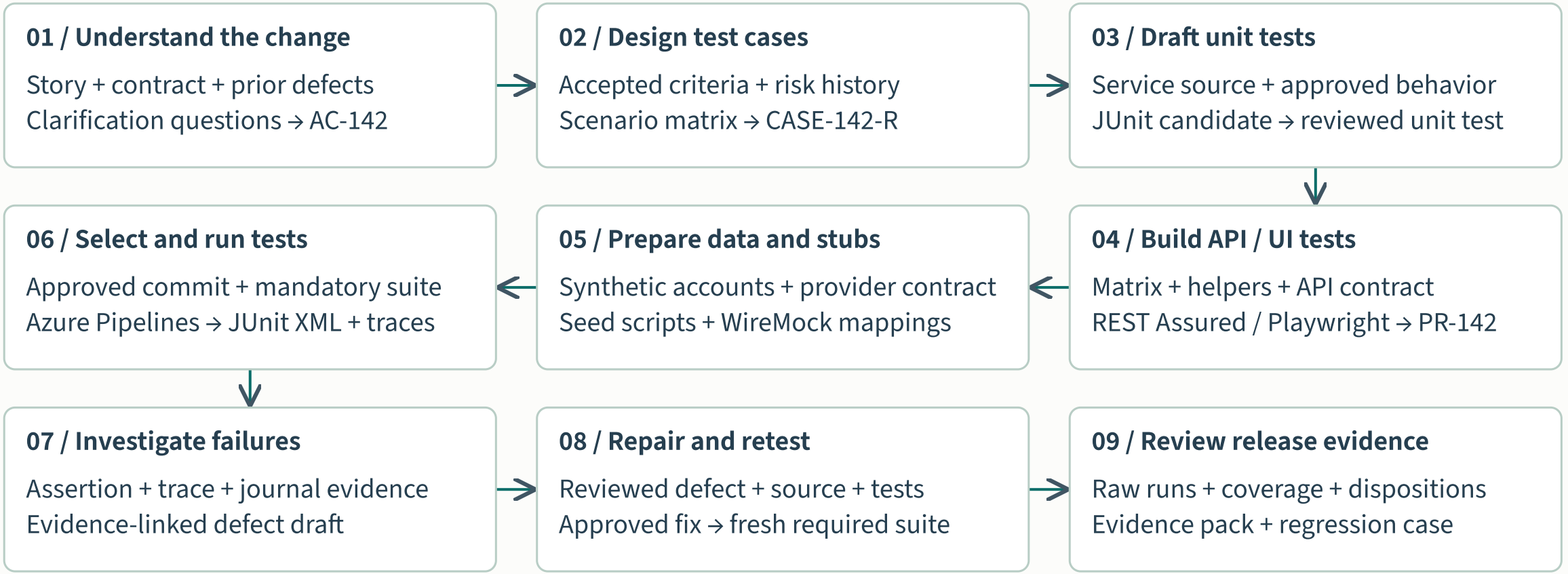
Compare accepted outcomes · Include unsuccessful attempts, review, rework and operating costs

Proposed evaluation design informed by ANZ's study limitations and METR's selection findings. Separate modernization gains from incremental AI effects.

Record every attempt and separate active effort, wait time and setup. Quality and net effort both inform whether to expand.

OUR BANKING CLIENT / AUTHORED PAYMENT SCENARIO

AI-assisted payment QE: the end-to-end workflow



AI assists the testing team. For code, fixtures, failure evidence and release handoffs, open the banking engineering walkthrough →